Planning in AI is the process of deciding a sequence of actions that an intelligent agent must perform to move from an initial state to a desired goal state.

Search	Planning
i. Search in AI is the process of exploring possible states on paths in a problem space to find a solution	i. Planning in AI is the process of deciding a sequence of actions that an agent must perform to go desired state
ii. find a path from initial state to goal	ii. Generates a sequence of actions to reach a goal.
iii. Requires limited knowledge	iii. Requires detailed domain knowledge
iv. focuses on exploring state space	iv. focuses on choosing and ordering actions
v. Uses state-space graph or tree representation	v. Uses symbolic action models like PDDL, STRIPS
vi. low level, algorithmic process	vi. High level, logical reasoning
vii. Transitions are implicit	vii. Transitions are explicit
viii. Minimal domain knowledge	viii. Rich domain knowledge
ix. Algorithm: BFS, DFS, A*, IDA*	ix. classical planning, HTN, SAT planning
x. Used in: Pathfinding, puzzles, games	x. Used in: Robotics, scheduling

Scanned with CamScanner

2

AI Planning Techniques

I. Goal Stack Planning

- Breaks the main goal into subgoals
- Uses a stack to push goals and pop actions
- Solves goals one by one

exam: Achieve goal → break into subgoals → apply actions

II. Forward Planning / Forward state-space search

- Starts from the initial state
- applies actions to progress toward the goal
- easy to implement

III. Backward Planning / Backward state-space search

- Starts from the goal state
- works backward to find actions that can achieve it
- Good when the goal is specific

IV. Hierarchical Task Network planning (HTN)

- Breaks a large task into smaller tasks
- Uses a hierarchy of tasks/subtasks

exa: plan a trip → tickets → luggage → hotel → etc

V. Stanford Research Institute Problem Solver (STRIPS)

- Represents actions with
  - Preconditions
  - add effects
  - delete effects
- classical planning framework

Scanned with CamScanner

VI. Partial-order Planning

- Does not require actions to be strictly ordered
- allows actions to be done in any order, as long as constraints are satisfied
- Produces a flexible plan

Detailed Goal Stack Planning

- Goal stack planning is one of the earliest planning techniques in AI
- It works backwards from the goal state toward the initial state, breaking the goal into smaller subgoals
- To achieve a goal, we push the preconditions of that goal onto a stack
- each precondition may have its own preconditions which must be solved first
- Continue pushing goals and subgoals on the stack and solving them step by step
- when all subgoals are satisfied, we reach a sequence of actions that transforms the initial state → goal state.
- the stack helps us keep track of goals, subgoals, and required actions in correct order.

Scanned with CamScanner

Goal Stack Planning Algorithm

1. Initialize: Push the goal state onto the stack
2. Decompose Goal: Push each predicate/subgoal of the goal state onto the stack
3. Repeat until the stack is empty
   - I- Pop the top element E from the stack
   - II- If E is a predicate/subgoal:
     - If E is already true → Do nothing
     - otherwise
       - → push the action that achieves predicate E onto the stack
       - → push the preconditions of that action onto the stack
   - III- If E is an action
     - apply the action to the current state
     - add the action to the plan

The Blocks World (AI Planning)

The blocks world is a classic environment used to teach and demonstrate planning algorithms.

It contains:

- A robot arm
- several wooden blocks with labels (A, B, C, etc)
- A table on which some blocks rest

Blocks can be stacked on top of each other or placed separately on the table.

Planning tasks involve transforming an initial state into a goal state by choosing the correct sequence of actions.

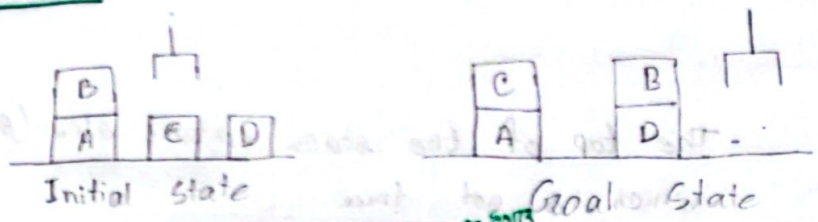
Scanned with CamScanner

5

Robot Arm capabilities

Operators	Precondition	Delete	Add
Stack (x,y)	clear(y) ∧ holding(x)	clear(y) Holding (x)	armempty on (x,y)
unstack (x,y)	armempty ∧ on(x,y) ∧ clear(x)	armempty ∧ on (x,y)	holding (x) ∧ clear(y)
Pickup(x,y)	clear (x) ∧ ontable (x) ∧ armempty	ontable (x) ∧ armempty	holding (x)
Put Down (x)	holding (x)	holding (x)	on table (x) ∧ armempty

Example



Initial state: ON (B, A) ∧ ONTABLE (C) ∧ ONTABLE (A) ∧ ONTABLE (D) ∧ CLEAR(B) ∧ CLEAR (C) ∧ CLEAR(B) ∧ ARMEMPTY

Goal state: ON(C, A) ∧ ON(B, D) ∧ ONTABLE (A) ∧ ONTABLE (D) ∧ CLEAR(C) ∧ CLEAR(B) ∧ ARMEMPTY

Solution

Goal stack

ON (C,A)

ON (B,D)

ON (C, A) ∧ ON (B,D)

→ push the original compound goal along with its sub goals on to the stack

Goal state এ যেগুলো initial state এ নাই শুধু সেগুলো নিয়ে কাজ শুরু করব

The following goals are already true in the initial state:

ONTABLE (A) ∧ ONTABLE (D) ∧ CLEAR(C) ∧ CLEAR(B) ∧ ARMEMPTY

The only goals to solve: ON (C,A) and ON(B,D)

Scanned with CamScanner

6

Goal Stack:

Stack (C,A)
On (B,D)
On (C,A) ∧ On (B,D)

To solve on(C,A) we can apply stack (C,A) action. So replace it

clear (A)
Holding (C)
Clear(A) ∧ holding(C)
stack (C,A)
On (B,D)
On (C,A) ∧ On(B,D)

stack(C,A) can be applied if its preconditions are true. So add its preconditions on the stack

On (B,A)
Clear (B)
armempty
on (B,A) ∧ Clear(B) ∧ armempty
unstack (B,A)
holding (C)
Clear (A) ∧ holding (C)
stack (C,A)
On (B,D)
On (C,A) ∧ On(B,D)

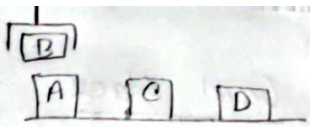
- The top of the stack contains clear(A) which is not true.
- So, to make the state true, replace clear(A) by unstack(B,A) and write the preconditions of unstack action

unstack (B,A)
holding (C)
Clear (A) ∧ holding (C)
stack (C,A)
On (B,D)
On (C,A) ∧ On (B,D)

on(B,A), clear(B) and armempty are all true, so pop these along with its compound goal.

Scanned with CamScanner

Goal Stack



holding (C)

clear(A) ∧ holding (C)

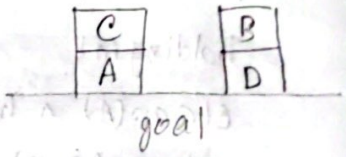
stack (C,A)

on (B,D)

on (C,A) ∧ on (B,D)

Add unstack (B,A) to the queue of a sequence of actions

Queue = unstack (B,A)



holding (C)

clear(A) ∧ holding(C)

stack(C,A)

on (B,D)

on (C,A) ∧ on (B,D)

now the top of the stack contains holding (C), which is not true

holding (B)

clear (D)

clear (D) ∧ holding (B)

stack (B,D)

holding (C)

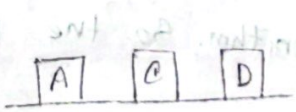
clear(A) ∧ holding (C)

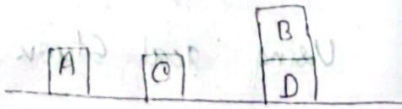
stack (C,A)

on (B, D)

on (C,A) ∧ on (B,D)

To make the state true use the operator stack (B,D). So push stack(B,D) along with its preconditions on to the stack.





holding (C)

clear (A) ∧ holding (C)

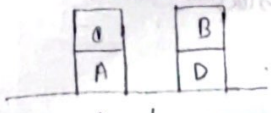
stack (C,A)

on (B, D)

on (C,A) ∧ on (B, D)

as all the preconditions are true, execute stack (B,D) and add it to the queue of sequence of actions

Queue: unstack (B,A), stack (B, D)



8

Goal Stack

ontable (c) ∧ clear (c) ∧ armempty

pickup (c)

holding (c)

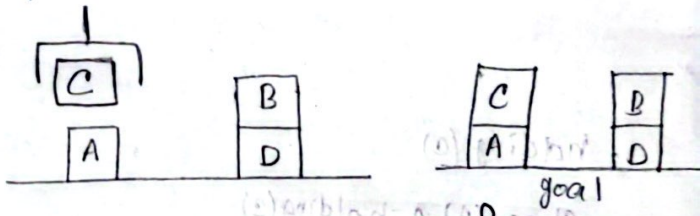
clear(A) ∧ holding (c)

stack (C, A)

on (B, D)

on (C, A) ∧ on (B, D)

- The top of the stack holding (c) is not true, so make the state ~~ter~~ true use the action pickup(c) and write the preconditions on to the stack



Holding (c)

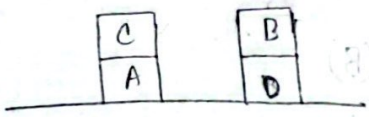
clear(A) ∧ holding(c)

stack (C, A)

on (B, D)

on (C, A) ∧ on (B, D)

- Here all the preconditions of pickup operaton are true, so add pickup (c) to the queue of a sequence of action.
- Queue = Unstack (B, A), stack (B, D), pickup (c)



holding (c)

clean (A) ∧ holding (c)

stack (C, A)

on (B, D)

on (C, A) ∧ on (B, D)

- Here all the preconditions of stack(C,A) is true, so add stack (C, A) to the queue
- Queue = unstack (B, A), stack (B, D), pickup (c), stack (C, A)

Using goal stack algorithm. So the plan for the given problem is

I. UnStack (B, A)

II. Stack (B, D)

III. PickUp (C)

IV. Stack (C, A)

Scanned with CamScanner

AI. logical Agents.pdf

Logical Agents are AI systems that use logic to represent knowledge and reasoning to make decisions

Representation refers to how knowledge about the world is stored
Reasoning refers to how ~~that~~ knowledge is used to make decisions
A Knowledge Based Agent is an AI agent that stores facts/information about the world in a knowledge base and uses logical reasoning to decide what actions to take.

A Problem Solving Agent is an AI agent that searches for solutions to specific problems without using general knowledge about the world.

Atomic Representations means representing knowledge where each fact/state is indivisible and can't be broken into smaller parts.

Knowledge Based Agents	Problem Solving Agents
I. Use a knowledge base to store facts about the world	I. Do not use a knowledge base
II. Use reasoning/inference to derive new knowledge	II. Use search to find a path to the goal
III. Have general world knowledge	III. Have very limited specific knowledge
IV. Flexible and can handle many situations	IV. Only solve the given problem
V. Work with logical representations	V. Work with atomic/indivisible representation
Exam: medical diagonosis system	ex: 8-puzzle solving agent

Scanned with CamScanner

Knowledge Base (KB)

- A knowledge base is a collection of sentences / facts that describe information about the world.
- These sentences are written in a knowledge representation language (like propositional logic, first-order logic)
- The KB allows an agent to store, retrieve and reason about knowledge

Inference engine	← Domain-independent algorithms
KB	← Domain-specific content

→ Stench, Breeze, Bump, Scream, Glitter

A Generic Knowledge Based Agent is an agent that learns from percepts, reasons to choose actions, and records those actions in its knowledge base for future use.

The Wumpus World

~ is a classic problem environment in Artificial intelligence used to test logical reasoning of agents.

- It is a cave made of inter-connected rooms arranged in a grid
- A dangerous creature called the wumpus hides in one room and will eat anyone who enters
- the agent has only one arrow to shoot and kill the wumpus
- Some rooms contain bottomless pits that trap anyone who enters
- One room contains a heap of gold, which is the agent's reward.

Stench		Breeze	Pit
	Breeze Stench Gold	Pit	Breeze
Stench		Breeze	
Start	Breeze	Pit	Breeze

Scanned with CamScanner

11

Actuators are the actions the agent can perform in the environment.

1. Forward, TurnLeft by 90°, TurnRight by 90°
   - the agent dies if it moves into a room with pit or a live wumpus
   - No movement occurs if the agent bumps into a wall

2. Gnab
   - Picks up the gold if it is in the same room as the agent

3. Shoot
   - Fires an arrow in the direction the agent is facing
   - The arrow continues until it hits either the wumpus or a wall

4. Climb
   - Climb out of the cave if the agent is at the starting position [1,1]

Sensors : ~~Up~~, works Up, down, left, Right. In digonally it not work

- Stench : Detected in squares directly adjacent to the wumpus
- Breeze : " " " " " a pit
- Glitter : " " " where the gold is located
- Bump : " when the agent walks into a wall
- Scream : Heard ~~when~~ when the wumpus is killed.

~~Pre~~

Percepts are the 5-value sensory inputs the agent receives in each step, represented as [Stench, Breeze, Glitter, Bump, Scream]

Scanned with CamScanner

12

Wumpus world characteristics

Deterministic : Actions have predictable outcomes

Discrete : 4×4 grid, step by step moves

static : Wumpus, pits, and gold don't move

single agent : only one agent operates

Partially observable : The agent can't see the whole environment, pit and wumpus locations are unknown initially.

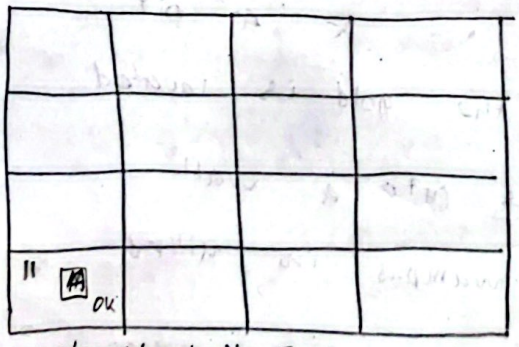
Stench		Breeze	Pit
	Breeze Stench Gold	Pit	Breeze
Stench		Breeze	
Start	Breeze	Pit	Breeze

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
1,1 OK	2,1	3,1	4,1

No Breeze, No Stench

~~(R 1) ¬S11 → ¬W11 ∧ ¬W12 ∧ ¬W21~~

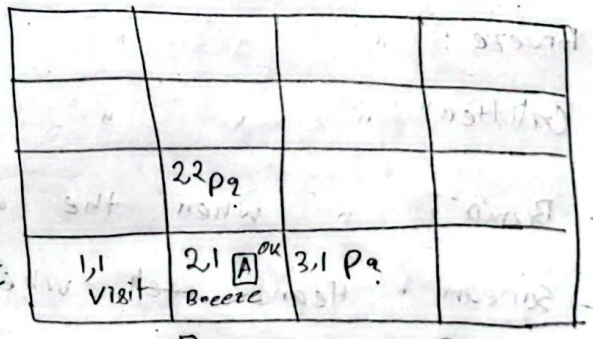
To Find Wumpus



No Stench, No Breeze

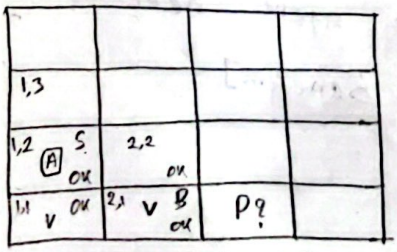
(R1) ¬S11 → ¬W11 ∧ ¬W12 ∧ ¬W21

(এর মানে কী → এই ঘর গুলোতে wumpus নাই)



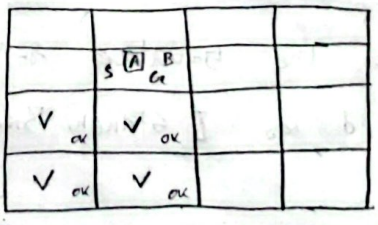
Breeze, so go Back

(R2) ¬S21 → ¬W11 ∧ ¬W21 ∧ ¬W22 ¬W31



(R3) ¬S12 → ¬W11 ∧ ¬W12 ∧ ¬W22 ∧ ¬W13

যদি 12 তে Stench না থাকতো তবে এটা হতো



(R4) S12 → W13 V W12 V W22 V W11

(এই ঘর গুলোতে wumpus থাকতে পারে)

যেহেতু 12 তে Stench আছে

How we can prov wumpus is in the room (13) using propositional rules

- Apply modus ponens to ¬S21 and R2

(R2) ¬S21 → ¬W11 ∧ ¬W21 ∧ ¬W22 ∧ ¬W31 ℓ ¬S21

¬W11 ∧ ¬W21 ∧ ¬W22 ∧ ¬W31

apply and elimination to this we get : ¬W21, ¬W22, ¬W31

- Apply modus ponens to S12 and R4

(R4) S12 → W3 v W12 v W22 v W11

S12

W13 v W12 v W22 v W11

- Apply unit resolution on W13 v W12 v W22 v W11 and ¬W11

W13 v W12 v W22

- Again apply unit resolution with W13 v W12 and ¬W12

W13

proved

Scanned with CamScanner

14

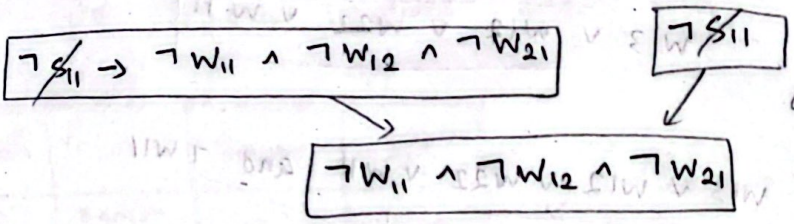
(R_1) $\neg S_{11} \rightarrow \neg W_{11} \wedge \neg W_{12} \wedge \neg W_{21}$

(R_2) $\neg S_{21} \rightarrow \neg W_{11} \wedge \neg W_{21} \wedge \neg W_{22} \wedge \neg W_{31}$

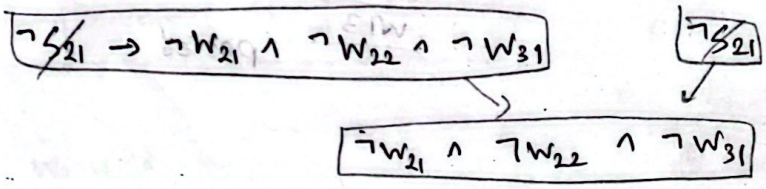
(R_3) $\neg S_{12} \rightarrow \neg W_{11} \wedge \neg W_{12} \wedge \neg W_{22} \wedge \neg W_{13}$

(R_4) $S_{12} \rightarrow W_{13} \vee W_{12} \vee W_{22} \vee W_{11}$

1. Apply modus ponens with $\neg S_{11}$ & R_1



2. Apply modus ponens with $\neg S_{21}$ & R_2



3. Apply modus ponens to S_{12} & R_4

$S_{12} \rightarrow W_{13} \vee W_{12} \vee W_{22} \vee W_{11}$ $\quad S_{12}$

$W_{13} \vee W_{12} \vee W_{22} \vee W_{11}$

4. Apply unit resolution

$W_{13} \vee W_{12} \vee W_{22} \vee W_{11}$ $\quad \neg W_{11}$

$W_{13} \vee W_{12} \vee W_{22}$

5. Apply unit resolution

$W_{13} \vee W_{12} \vee W_{22}$ $\quad \neg W_{22}$ ← এখানে wumpus নাই, তাই মিথ্যা

$W_{13} \vee W_{12}$

6. Apply unit resolution

$W_{13} \vee W_{12}$ $\quad \neg W_{12}$

W_{13}

∴ it proved that the wumpus is in room [1,3]

Scanned with CamScanner

logic is the systematic study of rules of inference and a formal language for representing information so that conclusions can be drawn

Syntax → the structure of well-formed sentences. $x+y=4$ → valid, $xy+=$ invalid

Semantics → the meaning of sentences, defining whether they are true or false in possible worlds. $x+y=4$ if $x=2, y=2$ then true

Entailment means that one statement logically follows from another.
if α entails β, than whenever α is true, β must also be true in all possible case.

Propositional Logic

Atomic Sentence : is the simplest kind of logical statement consisting of a single propositional symbol with no logical connectives.
example : P, Q, R, W_1, 3, Facing East

2. special proposition Symbols

1. Tautology : A statement that is true in all situations
2. Contradiction : " " " " false " " "

Complex Sentence have 5 logical connectives

1. ¬ (Not) : $\neg W_{1,3}$: atomic or negated atomic sentence
2. ∧ (and) : $W_{13} \wedge P_{31}$: called a conjunction, parts are conjuncts
3. ∨ (OR) : $(W_{13} \wedge P_{31}) \vee W_{22}$: called a disjunction. parts are disjuncts
4. ⇒ (implies) : $(W_{13} \wedge P_{31}) \Rightarrow \neg W_{22}$: called an implication
5. ⇔ (iff/Biconditional) : $W_{13} \Leftrightarrow \neg W_{22}$: True when both sides have the same truth value.

Scanned with CamScanner

P	Q	¬P	P∧Q	P∨Q	P⇒Q	P⇔Q
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

¬P : true if P is false

P∧Q : true only if both P and Q are true

P∨Q : true if at least one of P/Q is true

P⇒Q : false only when P is true and Q is false

P⇔Q : true when both have the same value

Implies (P ⇒ Q)	Biconditional (P ⇔ Q)
I. It means if P is true, then Q must be true	I. It means P and Q must have the same truth value
II. False only when P=true and Q=false	II. True when both are true or both false
III. One way relationship	III. two way relationship
IV. It also called implication	IV. It also called if and only if (IFF)
V. ex: If it rains, the ground gets wet	V. You pass iff you score ≥40

17

$\alpha \Rightarrow \beta$: If it rains, then the university closes.

Rain(α)	University closes (β)	$\alpha \Rightarrow \beta$ (if rain then close
T (Rain)	T (close)	T
T (Rain)	F (Doesn't close)	F
F (No Rain)	T	T
F	F	T

$\alpha \Leftrightarrow \beta$: If rains if and only if the university closes

If it rains, the university must close

If the university closes, that means it rains

They always happen together - either both true or both false

Rain (α)	University closes (β)	$\alpha \Leftrightarrow \beta$ (if rain then close
T (Rain)	T (close)	T
T	F (Doesn't close)	F
F (No Rain)	T	F
F	F	T

$\alpha \Rightarrow \beta$: Rain guarantees closure, but closure may occur for other reasons too

$\alpha \Leftrightarrow \beta$: Rain and closure always occur together - one happens exactly when the other does.

is true.

With 7 symbols, there are $2^7 = 128$ possible models, in three of these KB

Propositional Theorem Proving

$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$ commutativity of $\wedge$

$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$ " " $\vee$

$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$ associativity of $\wedge$

$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$ " " $\vee$

$\neg(\neg\alpha) \equiv \alpha$ double-negation elimination

$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$ contraposition

$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$ implication elimination

$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$ biconditional elimination

$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ De Morgan

$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$ De Morgan

$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$ distributivity of $\wedge$ over $\vee$

$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$ " " $\vee$ " $\wedge$

* Two sentences α and β are logically equivalent if they are true in exactly the same models

$$\alpha \equiv \beta \iff (\alpha \models \beta \text{ and } \beta \models \alpha)$$

Validity: A sentence is valid if it is true in all possible models such sentences are called tautologies. ex: $P \vee \neg P$ is always true

Deduction Theorem: For any sentences α and β, $\alpha \models \beta$ if and only if the sentence $(\alpha \Rightarrow \beta)$ is valid.

Satisfiability: A sentence is satisfiable if it is true in at least one model, even if not in all models.

ex: The KB $(R1 \wedge R2 \wedge R3 \wedge R4)$ is satisfiable because there are models where all of these are true.

Scanned with CamScanner

Inference and Proofs

Modus ponens: whenever any sentences of form $\alpha \Rightarrow \beta$ and α are given, then the sentence β can be inferred.

$$\text{Modus Ponens} = \frac{\alpha \Rightarrow \beta, \ \alpha}{\beta}$$

If (today is Tuesday) $\Rightarrow$ (then John will go to campus)

$$\text{MP} = \frac{\text{Today is tuesday}}{\text{therefore, Johan will go to campus}}$$

And Elimination: which says that from a conjunction, And-elemination any of the conjuncts can be inferred

$$\frac{\alpha \wedge \beta}{\alpha}$$

(Wumpus Ahead $\wedge$ Wumpus Alive), Wumpus Alive can be inferred

$$\text{Biconditional-Elimination} = \frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)} \quad \text{and} \quad \frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

Name	Rule	Tautology
$\frac{P \Rightarrow Q \quad P}{\therefore Q}$ Modus Ponens (law of Detachment)	$\frac{P \to Q \quad P}{\therefore Q}$	$((P \to Q) \wedge P) \Rightarrow Q$
Modus Tollens	$\frac{P \to Q \quad \neg Q}{\therefore \neg P}$	$((P \to Q) \wedge \neg Q) \Rightarrow \neg Q$
Hypothetical Syllogism (Transitivity)	$\frac{P \to Q \quad Q \to R}{\therefore P \to R}$	$((P \to Q) \wedge (Q \to R)) \Rightarrow (P \to R)$
Disjunctive Syllogism	$\frac{P \vee Q \quad \neg P}{\therefore Q}$	$((P \vee Q) \wedge \neg P) \Rightarrow Q$
Addition	$\frac{P}{\therefore P \vee Q}$	$P \Rightarrow P \vee Q$

Scanned with CamScanner

Simplification	$\frac{p \wedge q}{\therefore p}$	$(p \wedge q) \Rightarrow p$
Conjunction	$\frac{p \quad q}{\therefore p \wedge q}$	$(p) \wedge (q) \Rightarrow (p \wedge q)$
Resolution	$\frac{p \vee q \quad \neg p \vee r}{\therefore q \vee r}$	$((p \vee q) \wedge (\neg p \vee r)) \Rightarrow (q \vee r)$

1,4	2,4 pit	3,4	4,4
1,3 W?	2,3 A S G B	3,3 pit	4,3
1,2 S OK	2,2 OK	3,2	4,2
1,1 OK	2,1 B OK	3,1 pit	4,1

প্রত্যেক ঘর ৪ টা থাকতে হবে

(R1) : $\neg P_{1,1}$

(R2) : $B_{1,1} \Leftrightarrow (P_{12} \vee P_{21})$

(R3) : $B_{21} \Leftrightarrow (P_{11} \vee P_{22} \vee P_{31})$

(R4) : $\neg B_{11}$

(R5) : B_{21}

KB = {R-1, R-2, R-3, R-4, R-5}

Prove $\neg P_{12}$, that is, there is no pit in [1,2]

1. Apply biconditional elimination to R_2 to obtain

R_6 : $B_{11} \Rightarrow ((P_{12} \vee P_{21}) \wedge ((P_{12} \vee P_{21}) \Rightarrow B_{11})$

2. Apply and-elimination to R_6 to obtain

R_7 : $((P_{12} \vee P_{21}) \Rightarrow B_{11})$

3. Logical equivalence for contrapositives gives

R_8 : $(\neg B_{11} \Rightarrow \neg (P_{12} \vee P_{21}))$

4. Apply Modus Ponens with R_8 and the percept R_4 to obtain

R_9 : $\neg (P_{12} \vee P_{21})$

5. Apply De Morgans rule, giving the conclusion

R_{10} : $\neg P_{12} \wedge \neg P_{21}$

that is, neither [1,2] nor [2,1] contains a pit.

So 1,2 is no pit

[proved]

Scanned with CamScanner

21

Resolution is a rule of inference used to determine whether a set of clauses in CNF is satisfiable or unsatisfiable.

Conjunctive Normal Form = CNF

$$\text{Resolution rule} = \frac{\neg\alpha \Rightarrow \beta,\ \beta \Rightarrow \gamma}{\neg\alpha \Rightarrow \gamma}$$

$$\text{" " } = \frac{\alpha \vee \beta,\ \neg\beta \vee \gamma}{\alpha \vee \gamma}$$

* Summer ∨ winter, ¬winter ∨ cold

Using resolution = Summer ∨ cold

$$\frac{P \vee \neg Q \vee R,\quad \neg P \vee Q}{\neg Q \vee Q \vee R}$$ দুটো বাদ দিলে বিচ্ছেদ থাকে

* Prove that 3,1 is a pit

1. R_{11} : $\neg B_{12}$

   R_{12} : $B_{12} \Leftrightarrow (P_{11} \vee P_{22} \vee P_{13})$

2. R_{13} : $\neg P_{22}$

   R_{14} : $\neg P_{12}$

3. Apply Biconditional elimination to R_3

   R_{15} : $P_{11} \vee P_{22} \vee P_{31}$

4. Apply resolvent

   R_{16} : $P_{11} \vee P_{31}$

5. Unit resolution

   R_{17} : P_{31}

if there a pit in [1,1] or [3,1] and its not in [1,1]. then it's in [3,1]

Scanned with CamScanner

Resolution is a proof technique used in propositional logic that follows these steps

1. Select two clauses that contain conflicting terms
   ex: one clause has A, the other has ¬A
2. Combine the two clauses by taking their remaining literals
3. Cancel out the conflicting terms
   ex: remove A an ¬A

The resulting clause is called the resolvent

example: P ∨ Q
¬P ∨ R

1. Conflicting terms: P and ¬P
2. Combine: P ∨ Q ∨ ¬P ∨ R
3. Cancel P an ¬P

example

1. If it is a pleasant day you will do strawberry picking
2. If you are doing strawberry picking you are happy

above statements can be written in propositional logic like this

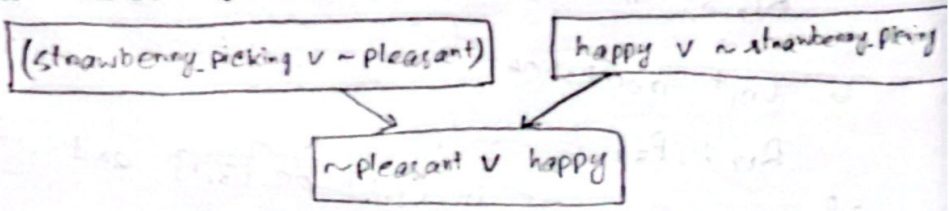
1. strawberry_picking ← pleasant
2. happy ← strawberry_picking

And again these statements can be written in CNF like this

1. (strawberry_picking ∨ ~pleasant) ∧
2. (happy ∨ ~strawberry_picking)

By resolving these two clauses and cancelling out the conflicting terms

3. ~pleasant ∨ happy



Scanned with CamScanner

Resolution

Rules

I. Convert the english statements into First order logic (FOPL)

II. Convert FoPL to CNF

III. Apply Negation to what's need to be proven

IV. draw Resolution Graph

Rules to convert FoPL to CNF — CNF only support ¬, ∨

1. $X \rightarrow Y \Rightarrow \neg X \vee Y$
2. $X \wedge Y \rightarrow Z \Rightarrow \neg(X \wedge Y) \vee Z \Rightarrow \neg X \vee \neg Y \vee Z$

Example

a. The humidity is high (P) or the sky is cloudy (Q)

b. If the sky cloudy (Q), then it will rain (R)

c. If the humidity is high (P), then it is hot (S)

d. It is not hot (S)

Proove : It will rain.

Solve

Step 1 : English to FoPL

a. Let P: humidity is high
Q: Sky is cludy
$P \vee Q$

b. Let Q: Sky is cludy
R: It will Rain
$Q \rightarrow R$

c. Let P: humidity is high
S: It is hot
$P \rightarrow S$

d. Let S: It is hot
$\neg S$

Step 2: FoPL to CNF

a. $P \vee Q \Rightarrow P \vee Q$

b. $Q \rightarrow R \Rightarrow \neg Q \vee R$

c. $P \rightarrow S \Rightarrow \neg P \vee S$

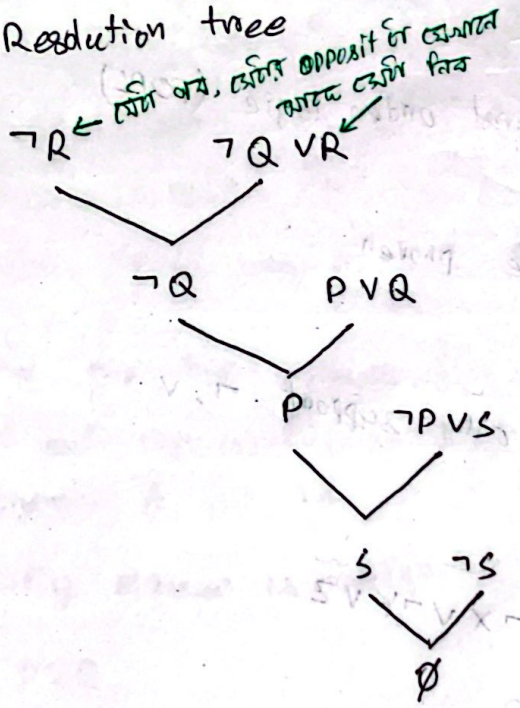
d. $\neg S \Rightarrow \neg S$

Step 3: Negative statement to be proven

It will Rain (R) $\Rightarrow \neg R$

Scanned with CamScanner

Step 4: Resolution tree



- Step 2, 3 এর প্রতিটাকে একবার use করতে পারবে।

⇒ Tree reached to null state which means ¬R is false or R is true

[proved]

example

Part I: English Sentences.

1. If it is sunny and warm day you will enjoy
2. If it is warm and pleasant day you will do strawberry picking
3. If it is raining then no strawberry picking
4. If it is raining you will get wet
5. It is warm day
6. It is raining
7. It is sunny.

Part II: Propositional statements

1. Sunny ∧ warm → enjoy
2. warm ∧ pleasant → strawberry_picking
3. raining → ¬ strawberry_picking
4. raining → wet
5. warm
6. raining
7. sunny.

Part III: CNF of Part II

1. (¬ sunny ∨ ¬ warm ∨ enjoy) ∧ [X ∧ Y ⇒ Z ⇒ ¬X ∨ ¬Y ∨ Z]
2. (¬ warm ∨ ¬ pleasant ∨ strawberry_picking) ∧
3. (¬ raining ∨ ¬ strawberry_picking) ∧ [X → Y ⇒ ¬X ∨ Y]
4. (¬ raining ∨ wet) ∧
5. (warm) ∧
6. (raining) ∧
7. (sunny)

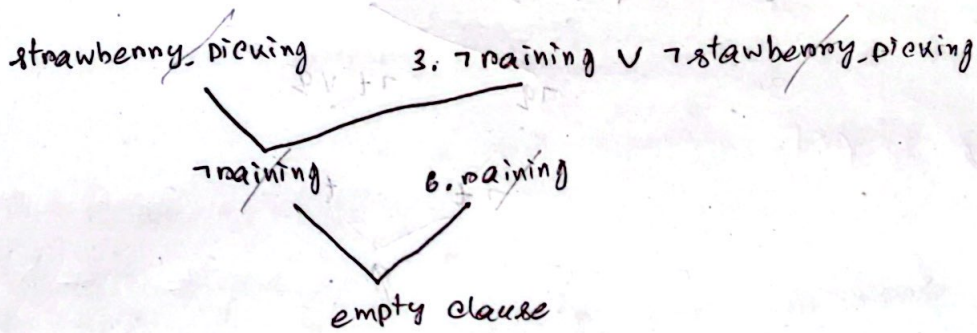
Scanned with CamScanner

25

Proved: You are not doing strawberry picking

Prove: ¬ strawberry_picking

assume: strawberry_picking (negate the goal and add it to given clauses)

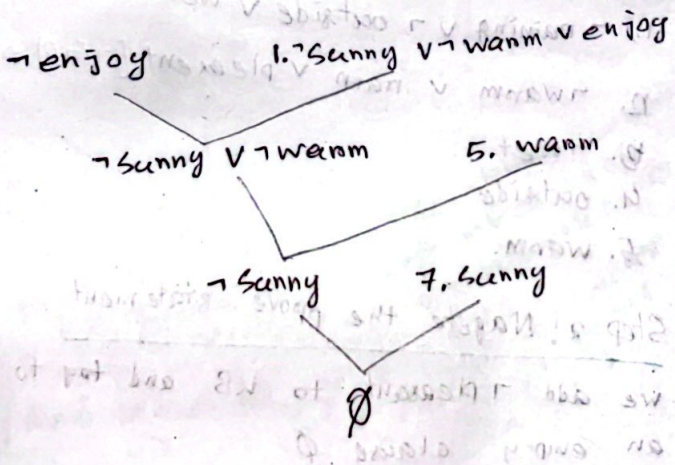


The tree reached to null state. which means strawberry_picking is false which means not strawberry picking is true [proved]

Goal 2: You will enjoy

prove: enjoy

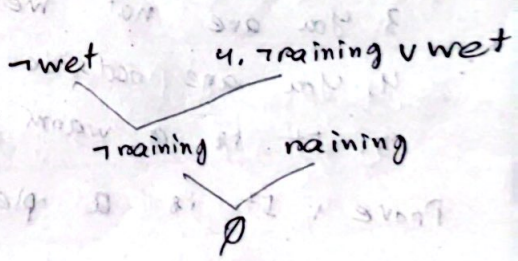
assume: ¬enjoy



goal 3: You will get wet

prove: wet

assume: ¬ wet



26

example

P

$(P \wedge q) \Rightarrow r$

$(s \vee t) \Rightarrow q$

t

to CNF

P

$\neg P \vee \neg q \vee r$

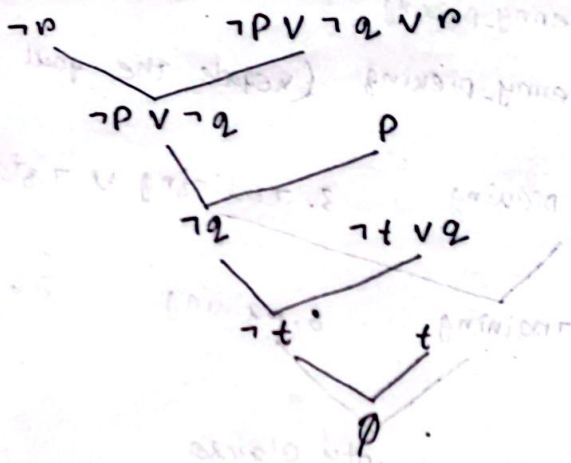
$\neg s \vee q$

$\neg t \vee q$

t

A resolution proof of r:

assum : $\neg r$



∴ r is true [proved]

example

1. If it is raining and you are outside then you will get wet
2. If it is warm and there is no rain then it is a pleasant day
3. You are not wet
4. You are outside
5. It is a warm day

Prove : It is a pleasant day.

Step 1 : Propositional statement

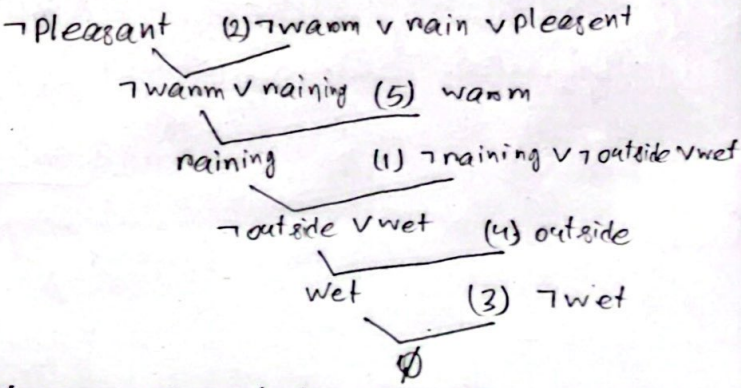
1. (raining ∧ outside) ⇒ wet
2. (warm ∧ ¬rain) ⇒ pleasant
3. ¬ wet
4. outside
5. warm

Step 2: CNF of step 1

1. ¬ raining ∨ ¬ outside ∨ wet
2. ¬ warm ∨ rain ∨ pleasant [¬(¬P) ⇒ P]
3. ¬ wet
4. outside
5. warm.

Step 3: Nagete the prove statement

we add ¬ pleasant to KB and try to denive an empty clause ∅



∴ It is a pleasant day [proved]

Constraint Satisfaction Problem (CSP)

- CSP consists of three components V, D, C
- V is set of variables $\{V_1, V_2, V_3 \dots V_n\}$
- D is " " domains $\{D_1\ D_2 \dots D_n\}$ one for each variable
- C " " constraints that specify allowable combination of values

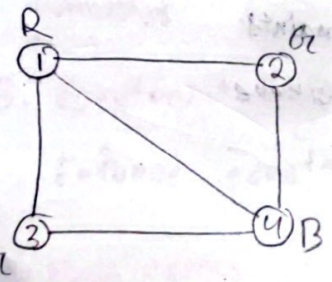
$C_i = (\text{Scope}, \text{relation})$

=> where scope is set of variable that participate in constraint
-> relation is relation that define the values that variable can take.

representation

V_1	V_2	C_1 say $V_1 \neq V_2$
Domain: A	B	$C_1 = ((V_1, V_2), (V_1 \neq V_2))$
(1,2)	(2,4)	$C_1 = (V_1, V_2), (A, B)$
		$C_1 = ((V_1, V_2), (1,2), (1,4), (2,4))$

example



$V = \{1, 2, 3, 4\}$

$D = \{\text{Red, Green, Blue}\}$

$C = \{1 \neq 2, 1 \neq 3, 1 \neq 4, 2 \neq 4, 3 \neq 4\}$

Use intelegent Backtracking
-> যেই node এ যেই conflict হবে, সেই node এ change করবে।
normal Backtracking এর মতো এক এক করে চেক করে না

	1	2	3	4
Initial Domain	R, Gr, B	R, Gr, B	R, Gr, B	R, Gr, B
1 = R	R	Gr, B	Gr, B	Gr, B
2 = Gr	R	Gr	Gr, B	B
3 = B	R	Gr	B (এখানে চেক করবে)	B (so Backtrack, conflict)
3 = Gr	R	Gr	Gr	B

28

A constraint satisfaction Problem (CSP) is a problem made up of

variables : things that need values

Domains : Possible values each variable can take

constraints: rules that specify which combinations of values are allowed

Type of Constraints in CSP

1. Unary Constraint
   - Affects only one variable
   - It restricts the value that a single variable can take

ex: in map-coloring, if SA ≠ Green, it means South Australia can't be green.

2. Binary Constraint
   - A constraint between two variable
   - It specifies allowed/disallowed combinations of values

ex: SA ≠ NSW., means SA and NSW cannot have the same colon.

3. Binary CSP
   - A CSP that includes only unary and binary constraints
   - It can be represented using a constraint graph, where

     nodes = variables

     edges = constraints between them.

4. Ternary Constraint
   - Affects 3 variables at the same time

ex: Between (X, Y, Z) means: X < Y < Z or X > Y > Z

So Y must lie between X and Z

5. Global Constraint
   - A constraint involving any number of variables, not just 1, 2 or 3
   - These are broad constraints that apply to many variables together

ex: - All-different $(x_1, x_2 \dots x_n)$ → all variables must have different values

- sum $(x_1, x_2, x_3) = 10$

Scanned with CamScanner

Real world example of CSPs

- Assignment problems: Deciding who will teach which class
- Timetabling : Scheduling classes, when and where they happen
- Hardware configuration: Choosing compatible components for a system
- Spreadsheets: Cells with formulas and constraints
- Transportation scheduling: planning bus/train/flight schedules
- Factory scheduling: Deciding when each machine does each job
- Floorplanning: arranging rooms or components without overlap

Different ways to Solve CSP

1. Basic Backtracking

Try values one by one, simple but slow. Not intelligent

2. Forward Checking

When a variable gets a value, remove conflicting values from future variables

3. Constraint Propagation/Arc Consistency

Enforce constraints to shrink domains before or during search

4. Heuristics

Uses smart strategies like MRV (minimum remaining values), degree heuristic or LCV (Least constraining value) to choose variables and values efficiently.

5. Local Search/Min Conflicts

Start with a complete assignment and fix conflicts, fast but may not guarantee a solution.

Scanned with CamScanner

30

Constraint Propagation is a CSP technique that reduces variable domains by enforcing constraints early, making the problem easier to solve

- It removes values that can't be part of any solution
- This reduces the number of choices during search
- It can be applied before search or during search
- Sometimes it can solve the whole CSP without backtracking

• Core idea behind constraint propagation is local consistency

Local consistency in CSP means that the values of a variable satisfy the constraints involving them, without considering the entire problem.

Type of Local Consistency

1. Node Consistency

- A variable is node consistent if all its values satisfy its unary constraints

ex: If SA can't be green, it domain {red, Green, blue} become {red, blue}

2. Arc Consistency

- A variable X_i is arc-consistent with X_j if every value in X_i has a compatible value in X_j that satisfies their binary constraint.

ex: For X an Y with $Y = X^2$, domains are reduced to compatible values
$X = \{0, 1, 2, 3\}$, $Y = \{0, 1, 4, 9\}$

3. Path Cons

- considers triples of variables ensuring that assignments to two variables can be extended to a third variable without violating constraints

ex: variable X, Y, Z with constraints $X < Y$ and $Y < Z$ are path consistent if all value assignments satisfy there constraints together

4. K-Consistency

- A CSP is K-consistent if any consistent assignment to $k-1$ variables can be extended to a kth variable without violating constraints.

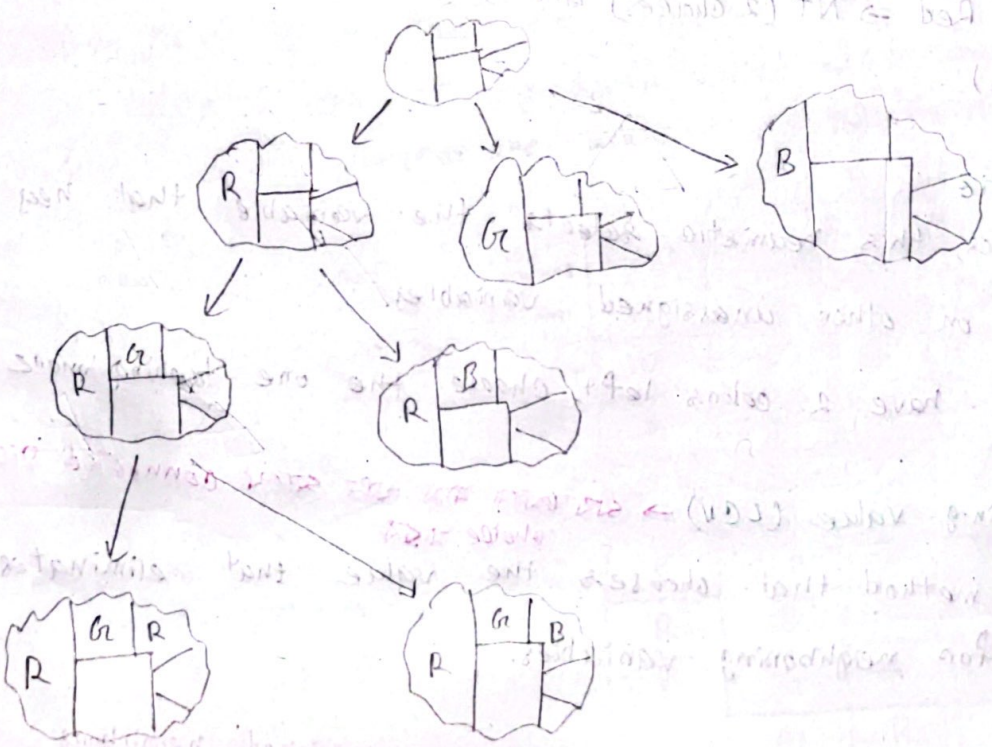
Scanned with CamScanner

Backtracking Search for CSP in AI

- Systematically assigns values to variables
- If a constraint is violated, it backtracks to try another value

Steps

- Select an unassigned variable
- pick a value from its domain
- check if it is satisfies all constraints
- If violated backtrack to previous variable
- repeat until a complete solution is found or all options exhausted



Variable and Value Ordering

1. Static or Random Ordering

is a variable selection strategy where variables are chosen in a fixed predefined order or random order without considering constrain/domain sizes.

2. Minimum Remaining values (MRV)

A heuristic that selects the variable with the fewest legal values left in its domain

ex: After WA = Red → NT (2 choices) and SA (2 choices) are chosen before Q (3 choices)

3. Degree Heuristic

when RMV ties, this heuristic selects the variable that has the most constraints on other unassigned variables

ex: If 2 regions have 2 colors left, choose the one touching more neighbors.

4. Least constraing value (LCV) → এমন রঙ বেছে নাও যাতে সবচেয়ে কম connected এলাকা choice করবে

A value-ordering method that chooses the value that eliminates the fewest options for neighboring variables.

example

lets say we have a node X connected to 5 unassigned neighbors

available colors: {Red, Green, Blue, Yellow}

Neighbors current available colors

N1 : {R, G, B}
N2 : {G, B, Y}
N3 : {R, B}
N4 : {R, G, Y}
N5 : {B, Y}

LCV calculation for node X

Red : remove from N1, N3, N4 → 3 constraints
Green : " " N1, N2, N4 → 3 "
Blue : " " N1, N2, N3, N5 → 4 "
Yellow : " " N2, N4, N5 → 3 "

∴ LCV choice : Red, Green or Yellow (all with 3 constraints)

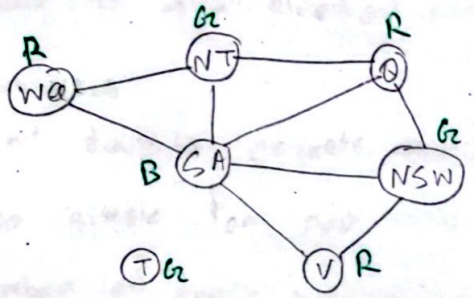
Scanned with CamScanner

Forward checking is a CSP inference technique that checks ahead after each variable assignment and removes values from future variables that would cause a conflict

- Forward checking improves basic backtracking by pruning future variable domains early.

Step

- Assign a value to a variable
- look ahead to unassigned neighboring variables
- remove/prune values from their domains that violate constraints
- check empty domains → if any domain become empty, backtrack immediately
- continue until all variables are assigned or backtracking is needed



	WA	NT	Q	NSW	V	SA	T
initial domains	R G B	R G B	R G B	R G B	R G B	R G B	R G B
WA = red	R	G B	R G B	R G B	R G B	G B	R G B
Q = green	R	B	G	R B	R G B	B	R G B
V = blue	R	B	G	R	B	Backtrack	R G B

now try Q = R

Q = R	R	G B	R	G B	R G B	G B	R G B
NT = G	R	G	R	G B	R G B	B	R G B
SA = B	R	G	R	G	R G	B	R G B
NSW = G	R	G	R	G	R	B	R G B
V = R	R	G	R	G	R	B	R G B
T = G	R	G	R	G	R	B	G

Final assignment

WA = R Q = R NT = G

NSW = G V = R SA = B

T = G

Advantages of forward checking

- reduce search space by removing invalid values early
- saves time by avoiding exploring impossible path
- finds dead-ends early
- avoiding wasted backtracking
- improves efficiency of backtracking search
- helps choose better future assignments
- reduces the chances of deep backtracking
- works well with heuristics like MRV, LCV, Degree heuristics
- keeps domains consistent after each assignment.

34

Min conflicts Heuristic is a local search method for csp where conflicts are fixed by choosing a conflicted variable and giving it the value that causes the fewest conflicts.

Scanned with CamScanner

First order logic.pdf

First order Logic (FOL) is a formal language used to describe facts, relationships, and rules about objects in a domain.

Propositional Logic is a branch of logic that deals with propositions and combines them using logical connectives like AND, OR, NOT, IMPLIES to form more complex statements.

Strengths of propositional logic

- Easy to understand because it uses only true/false statements
- Simple structure makes reasoning and proofs straightforward
- Can be checked by computer programs (SAT solvers) very efficiently
- Good for small, clear-cut problems with fixed facts.

Weaknesses

- Can't describe objects, relationships or quantifiers (like 'all', 'some')
- Too simple for real-world problems that need more detailed representation
- Number of truth combinations grows very large, making big problems hard to solve
- can't handle uncertainty, probabilities or incomplete information.

Key components

1. Constants: refer to specific objects in the domain. ex: John, Earth, Dhaka
2. Variables: Symbols that can represent any object in the domain. ex: x, y, z
3. Predicates:- Describe properties of objects or relationships between objects
   - always evaluate to true or false
   - Unary predicate: Property of a single object: John is a student: Student(John)
   - Binary " : relationship between two object: Loves(John, Mary): John loves Mary
   - Ternary " : " involving 3 objects: Between(A, B, C): B is between A and C

Scanned with CamScanner

4. Functions

- map objects to other objects in the domain
- they return an object, not true/false

ex: Father(John) → returns "Johns father"

type of functions

- Unary function: one input → one output ; Father(John)
- Binary " ; Two input → one output : Add(2,5) = 7
- Ternary " : Three input → one output : Max(x,y,z)

5. Quantifiers

Universal Quantifier (∀): states that a predicate is true for every element in the domain. ex: ∀x Student(x) → "Everyone is a student"

Existential Quantifier (∃): states that a predicate is true for at least one element in the domain. ex: ∃x Student(x) → "There exists at least one student."

6. Logical Connectives: Same as in propositional logic AND(∧), OR(∨), NOT(¬), implies(→), IFF(↔),

7. Terms: ~ are the basic building blocks used in FOL. constants, variables, function output.

Father(John), x, 5

8. Formulas: are logical statements formed from terms, predicates, quantifiers and connectives. Ex: ∀x (Student(x) → AttendsClass(x))

There are types of quantifiers

- Universal Quantification (all, any, each, every)
- Existential " (there exists, some, at least one)

Language	Ontological Commitment (what exists in the world)	Epistemological Commitment (what an agent believes about facts)
Propositional logic	Facts	true / false / unknown
First order logic	Facts, objects, relations	true / false / unknown
Temporal logic	facts, objects, relations, times	true / false / unknown
Probability theory	facts	degree of belief ∈ [0,1]
Fuzzy logic	facts with degree of truth ∈ [0,1]	known interval value

Scanned with CamScanner

3+

Alphabet of First-order Logic

logical symbols

- connectives : $\wedge, \vee, \Rightarrow, \Leftrightarrow, \neg$
- parenthesis : (,) and punctuation ,
- equality : =
- quantifiers : $\forall$ (universal), $\exists$ (existential)
- variables : $x, y, z \cdots, x_1, x_2 \cdots$

Non logical symbols

- constants : Socrates, turing, 1, earth--
- predicate symbols : true, false

  Father(x,y) // x is father of y

  Female(x) // x is female

- Function symbols :

  ged(x,y)

  FatherOf(x)

Terms

- constants : Socrates, turing ——
- variables : x, y, z ——
- functions : ged(x,y), FatherOf(x)

$f(x_1, x_2 \cdots x_n)$

function symbol — terms

Atomic Sentences

- Predicates : true, false

  Mother(A, B)

  Male(John)

- term equalities

  FatherOf(Apollo) = Zeus

An Atomic Sentence is the simplest kind of logical statement, it expresses a single proposition that can be either true or false and can't not be broken down int smaller parts

example

Loves(Alice, Bob) → Alice loves Bob

Person(Johan) → Johan is a person

Taller(Johan, Mary) → Johan is taller than Mary

A complex Sentence is a statement that is built from one or more atomic sentences combined with logical connectives or quantifiers.

$\forall x$ (Human(x) → Mortal(x)) : All humans are mortal

$\exists x$ (cat(x) ∧ Black(x)) : There exists a black cat

Human(Socrates) → Mortal(Socrates) : if Socrates is human, then Socrates is mortal

$\neg$(Loves(Alice, Bob) ∧ Loves(Bob, Alice)

: It is not true that Alice and Bob love each other mutually.

Scanned with CamScanner

English Sentences into FOL

1. One person is an ancestor of another if he is a parent of the other or he is an ancestor of an ancestor of the other

$\forall x \forall y (Parent(x,y) \vee \exists z (Ancestor(x,z) \wedge Ancestor(z,y))) \Rightarrow Ancestor(x,y)$

2. Every student has a father who is a professor. For every x, if x is a student, then Father(x) (a function) returns an object and professor(y) (a predicate) must be true.

$\forall x (Is\,Student(x) \rightarrow Is\,Professor(Father(x)))$

3. Everyone studying in HSTU is smart

$\forall x (Studies\,At(x, HSTU) \Rightarrow Smart(x))$

Smart(Rahim) if Rahim studies at HSTU

Smart(Karim) if Karim " " "

4. Someone studying in HSTU is smart

$\exists x (Studies\,At(x, HSTU) \wedge Smart(x))$

Common mistake to Avoid in FOL

1. $\Rightarrow$ is the main connective with $\forall$

Right: $\forall x (P(x) \Rightarrow Q(x))$

Wrong: $\forall x (P(x) \wedge Q(x))$

2. $\wedge$ is the main connective with $\exists$

Right: $\exists x (P(x) \wedge Q(x))$

Wrong: $\exists x (P(x) \Rightarrow Q(x))$

Scanned with CamScanner

5)

Properties of Quantifiers

1. Quantifiers of the same type commute

$\forall x \forall y\ P(x,y)$ is same as $\forall y \forall x\ P(x,y)$

$\exists x \exists y\ P(x,y)$ is " " $\exists y \exists x\ P(x,y)$

order doesn't matter

2. Quantifiers of Different types do not commute

$\exists x \forall y$ Loves (x,y) : There exists a person who loves everyone

$\forall y \exists x$ Loves (x,y) ! Everyone is loved by someone

two are completely different meaning $\exists x \forall y \neq \forall y \exists x$

3. Quantifiers Duality : Quantifiers flip under negation

$\forall$ become $\exists$, $\exists$ become $\forall$, Predicate gets negated

I. $\forall x$ Likes$(x,$ IceCream$)$: Everyone like ice cream

$\neg \exists x \neg$Likes $(x,$ IceCream$)$: There is no one who does not like ice creeem.

II. $\exists x$ Likes $(x,$ Broccoli$)$: someone likes broccoli

$\neg \forall x \neg$Likes $(x,$ Brocedi$)$! It is not true that everyone dislikes broccoli

Basic Quantified Statements

at least one x is P :	$\exists x P(x)$
all x are P :	$\forall x P(x)$
Some x are P :	$\exists x P(x)$
not all x are P :	$\exists x \neg P(x)$
no x are P :	$\forall x \neg P(x)$

Connections Between $\forall$ and $\exists$ using negation($\neg$)

Rule 1 : $\forall x \neg P(x) \equiv \neg \exists x\ P(x)$

'If everyone does not like P' is same

'There is no one who like P'

ex: $\forall x \neg$likes $(x,$ Parsnips$) \equiv \neg \exists x$ likes $(x,$ Parsnips$)$

noboy likes parsnips $\equiv$ it is not true that someone likes parsnips

Rule2! $\forall x\ P(x) \equiv \neg \exists x \neg P(x)$

if everyone is P $\equiv$ There is no one who is not P

ex: $\forall x$ Likes$(x,$ Ice cream$) \equiv \neg \exists x \neg$likes $(x,$ Icecream$)$

everyone likes ice cream $\equiv$ no one dislikes ice cream

Rule 3: De Morgan's rules for quantifiers

$\neg \forall x P(x) \equiv \exists x \neg P(x)$

not everyone is P $\equiv$ There exists someone who is not P

$\neg \exists x P(x) \equiv \forall x \neg P(x)$

There exists no x who is P $\equiv$ everyone is not P.

Scanned with CamScanner

Negation Inward / Flip Quantifiers

$\neg \forall \rightarrow \exists$

$\neg \exists \rightarrow \forall$

example:

R.H.S: $\neg \forall x \exists y \exists z \forall s \forall t \; P(x,y,z,s,t)$

$\exists x \neg \exists y \exists z \forall s \forall t \; P(x,y,z,s,t)$ $[\neg \forall \rightarrow \exists]$

$\exists x \forall y \neg \exists z \forall s \forall t \; P(x,y,z,s,t)$ $[\neg \exists \rightarrow \forall]$

$\exists x \forall y \forall z \forall s \forall t \; P(x,y,z,s,t)$

$\exists x \forall y \forall z \exists s \forall t \; P(x,y,z,s,t)$

$\exists x \forall y \forall z \exists s \exists t \; \neg P(x,y,z,s,t)$

= L.H.S

4|

4

1. Brothers are siblings : $\forall x \forall y\ (Brother(x,y) \leftrightarrow Sibling(x,y))$

   means : If someone is a brother of another person, then they are siblings

2. One's husband is one's male spouse : $\forall x \forall y\ (Husband(x,y) \leftrightarrow (Male(x) \wedge Spouse(x,y)))$

   = A husband is a spouse who is male

3. One's mother is one's female parent : $\forall m, c\ (Mother(c) = m \leftrightarrow (Female(m) \wedge Parent(m,c)))$

   $\forall m \forall c = \forall m, c$

   = A mother is a parent who is female

4. Sibling is symmetric : $\forall x, y\ (Sibling(x,y) \leftrightarrow Sibling(y,x))$

   = If x is a sibling of y, then y is also a sibling of x

5. Every dog is larger than some cat : $\forall x\ (D(x) \Rightarrow \exists y\ (C(y) \wedge Larger(x,y)))$

6. Alice has at least one cat : $\exists x\ (C(x) \wedge H(a,x))$

~~7. Alice has at most one cat : $\forall x \forall y\ ((C(x) \wedge H(a,x) \wedge \forall y\ ((C(y) \wedge H(a,y)) \rightarrow x=y))$~~

7. Alice has at most one cat : $\forall x \forall y\ ((C(x) \wedge H(a,x) \wedge C(y) \wedge H(a,y)) \rightarrow x = y)$

8. Alice has exactly one cat : $\exists x\ (C(x) \wedge H(a,x) \wedge \forall y\ ((C(y) \wedge H(a,y)) \rightarrow x = y))$

9. Alic has at least two cat : $\exists x \exists y\ (C(x) \wedge H(a,x) \wedge C(y) \wedge H(a,y) \wedge \neg(x = y))$

10. Alic has at most two cats

    $\forall x \forall y \forall z\ ((C(x) \wedge H(a,x) \wedge C(y) \wedge H(a,y) \wedge C(z) \wedge H(a,z)) \rightarrow (x=y \vee x=z \vee y=z))$

11. Alic has exactly two cats

    $\exists x \exists y\ (C(x) \wedge H(a,x) \wedge C(y) \wedge H(a,y) \wedge \neg(x = y) \wedge \forall z\ ((C(z) \wedge H(a,z)) \rightarrow (z=x \vee z=y)))$

12. Nobody loves Alice unless they are friend of Bob's

    $\forall x\ (loves(x, Alice) \rightarrow Friend(x, Bob))$

13. All red or Blue boxes contain a prize

    $\forall x\ ((Box(x) \wedge (Red(x) \vee Blue(x))) \rightarrow \exists y\ (Prize(y) \wedge Contains(x,y)))$

Scanned with CamScanner

14. There is a student who has read every book in the library

$\exists x (Student(x) \wedge \forall y ((Book(y) \wedge inLibrary(y)) \rightarrow HasRead(x,y)))$

15. Every person who owns a dog loves it

$\forall x \forall y ((Person(x) \wedge Dog(y) \wedge Owns(x,y)) \rightarrow Loves(x,y))$

16. There is a book that is read by ~~some~~ every student

$\exists x \forall y ((Book(x) \wedge Student(y)) \rightarrow Reads(y,x))$

17. Every person has a mother

$\forall x (Person(x) \rightarrow \exists y (Mother(y,x)))$

Inference in First order logic

KB - Knowledge Base

Inference is the process of deriving new information, conclusions or statement from existing known facts or premises using rules of logic.

Logical inference Problem:

Given a knowledge base KB (a set of sentences) and a sentences α, does the KB semantically entail α?

$KB \models \alpha$?

In propositional logic, inference can be done using 3 main method

1. Truth table Method: Uses truth tables to check if α is true in all interpretations where KB is true
2. Inference Rules: Applies formal rules (modus ponens, disjunction elimination, And/or elimination, Double negation) to derive α from KB
3. Resolution / SAT approach:
   - convert KB an $\neg\alpha$ into clauses (CNF)

Scanned with CamScanner

- Use resolution-refutation
  - if adding $\neg\alpha$ leads to contradiction $\rightarrow \alpha$ is entailed
  - if no contradiction $\rightarrow \alpha$ is not entailed

Unlike propositional logic, where every statement is either true or false,

First order logic is more expressive.

It allows reasoning about

- Variables (x, y, z)
- Predicates (Human(x), Loves(x,y))
- Quantifiers (∀ - for all, ∃ - there exists)
- Relations and Function

Feature	Propositional logic	First Order logic
1. Basic unit	Proposition (P, Q, R)	Predicates with argument (P(x), Loves(x,y))
expressiveness	less expressive	Highly
Variables	not allows	allowed (x, y —)
Quantifiers	not supported	supported (∀, ∃)
Internal structure of sentence	Not accessible	Accessible (Predicate, function, terms)
Knowledge representation	Simple Facts	complex fact, relationships, functions
Example	$P \wedge Q \rightarrow R$	$\forall x$ Human(x) → Mortal(x)
Domain	Fixed only true / false	can refer to a domain of objects
Complexity	Simple	complex
Application	simple logic, boolean reasoning	AI reasoning, natural language, databases, expert system.

Scanned with CamScanner

Inference Techniques in FOL

1. Resolution
   - Main inference method in FOL
   - convert KB to CNF and apply the resolution rule
   - Used in automated theorem proving

2. Forward chaining
   - start from known facts → derive new facts
   - Premises → conclusion

   ex: Human(x) → Mortal(x), Human(Sultan) ⇒ Mortal(Sultan)

3. Backward chaining
   - start from the goal → work backward to find supporting facts/rules
   - Used in prolog

4. Unification
   - Process of matching 2 predicates by substituting variables

   ex: Loves(x, John) and Loves(Mary, y) unify with x = Mary, y = John

Type of Sentence in FOL

1. Open sentence: A sentence that contains at least one free variable (not quantified)

   Loves(x, Bob) ; x is free

2. Bound variable: A variable that is inside the scope of quantifier (∀/∃)

   ∀x Loves(x, Bob) ; x is bound

3. Free variable: A variable not bound by any quantifier

   Loves(x, Bob) ; x is free

4. closed sentence: A sentence with no free variables. It can be true or false

   ~~Non-ground~~

   ∀x Loves(x, Bob)

Scanned with CamScanner

5. Non Ground closed sentence : Has only bounded variables but no free ones

$\forall x$ loves(x, Bob) ! x is bound, so it is closed but non-ground

6. Ground Sentence : Contains no variables only constants

Loves (Alice, Bob) : Both Alice and Bob are constants

- All ground sentences are closed but not all closed sentences are ground

Variable Substitution is the process of replacing a variable in a logical expression with term (a constant, function or another variable)

SUBST ({x/sam, y/Pam}, Likes (x, y)) = Likes (sam, Pam)

SUBST ({x/z, y/fatherof(John)}, Likes (x, y)) = Lives (z, fatherof (John))

Unification is the process of making two logical expressions identical by finding suitable substitution for their variables.

Loves (x, John) and Loves (Many, y)

Substitution , x = Many, y = John

∴ Loves (Many, John)

Formal Definition

unification takes two formulas f and g and returns substitution θ such that

Subst [θ, f] = Subst [θ, g]

1. Unify [Knows (alice, anithmetic), Knows (x, anithmetic)] = {x / alice}

2. Unify [Knows (alice, y), Knows (x, z)] = {x/alice, y/z}

alice → x

y → z

3. Unify [Knows (alice, y), Knows (bob, z)] = can't unify.

alice vs bob different constants.

4. Unify [Knows (alice, y), Knows (x, f(x))] = {x/alice, y/F(alice)}

Scanned with CamScanner

Universal Elimination / ∀ Elimination : allows to take a universally quantified statement (something true for all elements and apply it to a specific individual

$$\frac{\forall x\ \alpha}{\text{Subst}(\{v/g\}, \alpha)}$$

General form

$$\frac{\forall x\ \phi(x)}{\phi(a)}$$

1. ∀x Human(x) → Mortal(x) , substitute/infer x → Sultan Human(Sultan) → Mortal(Sul)
2. ∀x loves(x, IceCream) , x → Bob loves(Bob, icecream)
3. ∀x (King(x) ∧ Greedy(x) ⇒ Evil(x))

   x → John

   King(John) ∧ Greedy(John) ⇒ evil(John) , {x / John}

   ⇒ এভাবে substitute হবে

   {x / Father(John)}

   x → Father(John)

   King(Father(John)) ∧ Greedy(Father(John)) ⇒ Evil(Father(John))

Existential Elimination / ∃ - Elimination : allows to use a witness, a new constant symbol, to reason from an existential statement.

1. ∃x Loves(x, IceCream)

   introduce a new constant a

   ∴ Loves(a, IceCream)

2. ∃x (King(x) ∧ Greedy(x))

   introduce a new constant K

   King(K) ∧ Greedy(K)

3. ∃x Kill(x, victim)

   introduce a new constant Murderer

   Kill(Murderer, victim)

   ↳ Skolem constant

4. ∃x (Crown(x) ∧ OnHead(x, John))

   new constant C1

   Crown(C1) ∧ OnHead(C1, John)

   ↳ Skolem constant

General form

$$\frac{\exists x\ \phi(x)}{\phi(a)}$$

Scanned with CamScanner

A skolem constant is a new constant symbol introduced to eliminate an existential quantifier (∃) during logical transformations - especially in skolemization

Skolemization is ~~the~~ a process in first order logic used to eliminate existential quantifiers (∃) by introducing new skolem constants / functions

Universal Rules for Quantifiers

1. Universal Instantiation / elimination
   remove the universal quantifier and substitute the variable with any constant or term.

   ∀x Sister(Amy, x) ; instantiate x = Jane ∀x Sister(Amy, Jane)

   $\frac{\varphi}{\forall x \varphi}$: x is not free in φ

2. Existential Instantion
   If some specific instance is true, then we can introduce an existential quantifier.

   Likes (Ben, IceCream) generalize ∃x likes (x, IceCream)

   $\frac{\varphi(a)}{\exists x \varphi(x)}$ a - is a grounded term in φ
   x - is not free in φ

   $\frac{\alpha}{\exists v \text{ Subst}(\{g/v\}, \alpha)}$

Modus Ponens

Modus Ponens is a fundamental rule of inference in logic that states if p implies q and p is true, then q must also be true.

if Premises : $P_1, P_2 \dots P_k$
Rule : $P_1' \wedge P_2' \wedge \dots \wedge P_k' \rightarrow Q$
and there exists a unifier σ such that
$\sigma(P_i) = \sigma(P_i')$ for all i
then infer = σ(Q)

example

- Premises

Takes (alice, cs221)

Covers (cs221, mdp)

$\forall x \forall y \forall z$ Takes $(x, y) \wedge$ Covers $(y, z) \rightarrow$ Knows (x, z)

Unify with facts x/alice, y/cs221, z/mdp

$\therefore \theta = \{x/alice, y/cs221, z/mdp\}$

Apply θ to the conclusion

Knows $(x, z) \rightarrow$ Knowns (alice, mdp)

result : Knowns (alice, mdp)

2 fundamental strategies used to derive conclusions from a set of facts an rules

1. Forward chaining is an inference method that starts from known facts and applies rules to generate new facts until the goal is reached

start with facts → apply rules → produce new facts → repeat

Algo

- Start from the known facts
- Trigger all the rules whose premises are satisfied
- add their conclusions to the known facts
- Repeat steps 2 and 3 until one of the following situations occurs
  - The query is answered
  - No new facts are added

when to use

- many known facts at the start
- to automate derive all possible conclusions
- don't have specific goal at the beginning
- to keep inferring until no none new facts appear

ex: expert system, diagnosis system, monitoring system.

Scanned with CamScanner

2. Backward chaining is an inference method that starts from a goal and searches backward for rules and facts that can support/prove that goal

Algo

1. Start with the goal that need to prove
2. find rules that conclude that goal
3. Treat each rule's premises as sub-goal
4. Try to prove each sub-goal using facts / other rules
5. If all sub-goal are proven, the main goal is proven
6. If no rule can prove a sub-goal, the process fails.

When to use

- When start with goal and want to work backward
- want efficient reasoning for a single specific question
- knowledge base has many facts, but only few are needed for the goal

ex: Use in Prolog, goal-driven expert system

example

Facts KB

1. John is parent of Alice
2. Alice " " Bob
3. Bob " " Carol

Rules KB

1. If someone is a parent of another person, then they are also an ancestor of that person
2. If someone is a parent of another person and that second person is an ancestor of a third person, then the first person is also an ancestor of the third person

Goal: Prove that John is an ancestor of Carol

Solve

Translate FOL

Facts

1. Parent (John, Alice)
2. " (Alice, Bob)
3. " (Bob, Carol)

Rules KB

1. Parent $(x, y) \rightarrow$ Ancestor (x, y)
2. Parent $(x, y) \wedge$ Ancestor $(y, z) \rightarrow$ Ancestor (x, z)

Query: Ancestor (John, Carol)

Simple Forward chaining

Apply Rule 1 to each parent relationship

- Parent (John, Alice) → Ancestor (John, Alice)
  Parent (Alice, Bob) → " (Alice, Bob)
  Parent (Bob, Carol) → Ancestor (Bob, Carol)

Apply Rule 2 using the new ancestor fact

Parent (John, Alice) ∧ Ancestor (Alice, Bob)) → Ancestor (John, Bob)
Parent (Alice, Bob) ∧ Ancestor (Bob, Carol) → Ancestor (Alice, Carol)
Parent (John, Alice) ∧ Ancestor (Alice, Carol) → Ancestor (John, Carol)

∴ John is an ancestor of Carol.

51

Baeu word chaining

Goal: Ancestor (John, Carol)

1. Apply Rule 2 !

Ancestor (John, Cannol) ← Parent (John, x) ∧ Ancestor (x, Carol)

→ Parent (John, Alice) is true → new goal Ancestor (Alice, Carol).

2. Apply Rule 21

Ancestor (Alie, Carol) ← Parent (Alice, x) ∧ Ancestor (x, Carol)

Parent (Alie, Bob) is true → new goal Ancestor (Bob, Carol)

3. Use Rule 1!

Parent (Bob, Carol) → Ancestor (Bob, Carol)

Parent (Bob, Carol) is true → goal satisfied

∴ John is an ancestor of Carol.

Resolution is a rule of inference used in both propositional logic an first order logic to derive new clauses on prove contradiction

Animal (F(x)) V loves (G(x), x) (¬loves (u,v) V ¬kills (u,v))

resolvent: Animal (F(x)) V ¬kills (G(x), x)) ; θ={ u / G(u), v/x}

Binary Resolution is a rule of inference in propositional logic used to derive new clauses by eliminating complementary literals.

I. P(x) V Q(x) unfy P(x) with ¬P(a) → x = a

II. ¬P(a) V R(a) Result Q(a) V R(a)

Full Resolution allows resolving multiple pairs of matching literals in the same step.

1. $P(x) \vee Q(x) \vee R(x)$ ^ 11. $\neg P(a) \vee \neg Q(a) \rightarrow$

resolve $x \rightarrow a$

$P(a) \vee Q(a) \vee R(a)$ $\quad \neg P(a) \vee \neg Q(a)$

$R(a)$

* The law says that it is a crime for an American to sell weapons to hostile nations. The country nono, an emeny of America, has some missiles, and all of its missiles were sold to it by Colonel west, who is American.

To FOL:

1. It is a crime for an American to sell wepons to hostile nations

$\forall x$ American $(x) \wedge$ Wepon $(y) \wedge$ Hostile $(z) \wedge$ Sells $(x, y, z) \Rightarrow$ Criminal (x)

2. Nono — has some missiles

$\exists x$ Owns $(Nono, x) \wedge$ Missile (x)

3. All of its missiles were soled by Colonel west

$\forall x$ Missile$(x) \wedge$ Owns $(Nono, x) \Rightarrow$ Sells $(west, x, Nono)$

4. Need to know that missiles are weapons

Missile $(x) \Rightarrow$ weapon (x)

5. Also need to know that an enemy of America counts as hostile

Enemy $(x, America) \Rightarrow$ Hostile(x)

Scanned with CamScanner

5>

6. West, who is American
   American (west)

7. The country Nono, an enemy of America
   Enemy (Nono, America)

Prove : Criminal (west)
¬Criminal (west)

$P \wedge Q \rightarrow S$
$P \rightarrow S$
$Q \rightarrow S$

To CNF

1. ¬American (x) ∨ ¬weapon (y) ∨ ¬Hostile (z) ∨ sells (x,y,z) ∨ Criminal (x)
2. Own (Nono, x) ∧ Missile (x) → [illegible]
3. ¬Missile (x) ∨ ¬Owns (Nono, x) ∨ sells (west, x, Nono)
4. ¬Missile ∨ weapon (x)
5. ¬enemy (x, America) ∨ Hostile (x)
6. American (west)
7. enemy (nono, American)

¬criminal (west) ¬American (x) ∨ ¬weapon(y) ∨ ¬Hostile(z) ∨ sells (x,y,z) ∨ criminal (x)
{x/west}

¬American (west) ∨ ¬weapon (y) ∨ ¬Hostile(z) ∨ sells (west, y, z) American (west)

¬weapon (y) ∨ ¬Hostile(z) ∨ sells (west, y, z) ¬Missile (x) ∨ weapon (x)
{x/y}

¬Missile (y) ¬Hostile(z) ∨ ¬sells (west, y, z) Missile (y)

¬Hostile(z) ∨ sells (west, y, z) ¬Missile (x) ∨ ¬owns (Nono, x) ∨ sells (west, x, Nono)
{z/Nono}

¬Hostile (Nono) ¬Missile (x) ¬owns (Nono, x) owns (Nono, x)

¬Hostile (Nono) ¬Missile (x) Missile (x)

¬Hostile (Nono) ¬enemy (x, America) ∨ Hostile (x)
{x/Nono}

¬enemy (Nono, American) enemy (nono, American)

Scanned with CamScanner